

[23/09, 11:39 pm] S.....!: What is moving average filter which is used in this paper

[23/09, 11:40 pm] S.....!: In this paper, the moving-average filter is used specifically for step-distance estimation in pedestrian motion tracking.

Here is how it is applied based on the text:

1. Its Specific Purpose in the Paper

When estimating distance walked, the system uses a calibrated fixed step length as its baseline. However, raw step-by-step estimates can fluctuate erratically due to variations in walking pace, sensor noise, or stride irregularities. The moving-average operation is applied across successive step measurements to produce a smoother step-distance estimate over time.

2. Implementation Context (Handwritten Note)

The handwritten note at the top margin of your paper gives the exact windowing logic being used:

"looks at the current sample, the one before [before], & the one after, & averages th..."

This describes a 3-point centered moving-average filter:

[23/09, 11:42 pm] S.....!: Instead of relying solely on the instantaneous measurement of a single step, the filter averages the current step calculation ($x[n]$) with the preceding step ($x[n-1]$) and the succeeding step ($x[n+1]$) to eliminate sudden outliers.

[23/09, 11:46 pm] S.....!: Scikit-learn (often imported as `sklearn`) is a widely used open-source Python library for machine learning, data mining, and statistical modeling. Built on top of NumPy, SciPy, and Matplotlib, it provides efficient, high-level tools for implementing classic (non-deep-learning) machine learning algorithms.

Key Capabilities

Classification: Identifying which category an object belongs to (e.g., Decision Trees, Random Forests, Support Vector Machines, Logistic Regression).

Regression: Predicting continuous-valued outputs (e.g., Linear Regression, Ridge, Lasso).

Clustering: Grouping unlabeled data points automatically (e.g., K-Means, DBSCAN).

Dimensionality Reduction: Reducing the number of random variables to analyze (e.g., Principal Component Analysis / PCA).

Data Preprocessing & Evaluation: Standardization, feature scaling, categorical encoding, cross-validation, and performance metrics (confusion matrix, accuracy, F₁-score).

How Scikit-Learn Fits into Embedded AI / TinyML

Scikit-learn runs on standard computer environments (Python on desktop/servers) and cannot run directly on microcontrollers due to memory and OS constraints.

In TinyML workflows—such as the one described in the paper:

Offline Training: Raw sensor data (accelerometer/gyroscope) collected from hardware is processed on a PC using Scikit-Learn to train a compact machine learning model (e.g., a Decision Tree).

Translation/Export: The trained model parameters and decision rules (if-else branches) are extracted from Scikit-Learn.

Embedded Execution: These rules are translated into lightweight C/C++ or MicroPython code that executes standalone on a resource-constrained microcontroller (like the Arm Cortex-M7).

[23/09, 11:51 pm] S.....!: Offline activity classification means identifying human physical movements (such as stationary, walking, or running) using motion sensor data processed entirely on a local microchip—without sending data to cloud servers or requiring an active internet connection.

In the paper, the word "offline" operates in two key ways:

1. Offline Model Training (On a Computer)

Instead of forcing a low-power microcontroller to do heavy machine learning math, the model is built beforehand:

Raw movement data from accelerometers and barometers is recorded.

A Decision Tree model is trained on a PC using Scikit-Learn.

The resulting decision rules (a sequence of simple if-else thresholds) are exported.

2. Offline Model Execution (On the Microcontroller)

The exported decision rules are written in lightweight MicroPython and uploaded to the Arm Cortex-M7 processor:

The sensor collects data live on the device.

The chip instantly evaluates the data against the stored if-else rules.

The user's activity status is determined locally on the spot.

Key Advantages of Offline Processing

Low Latency: Eliminates the delay caused by sending data back and forth to the cloud.

Lower Power Consumption: Wireless radios (Wi-Fi, Cellular, Bluetooth) consume significantly more battery than running simple logic on an embedded processor.

Data Privacy: Raw physical motion data remains on the wearable device and is never exposed online.

Reliability: Functions in environments with no network coverage or GPS signals (e.g., inside buildings, basements, or remote locations).

[23/09, 11:53 pm] S.....!: Pedestrian Dead Reckoning (PDR) is a self-contained navigation technique used to track a walking person's position over time without relying on external signals like GPS. It works by starting at a known position and continuously calculating each new position step-by-step using relative sensor data.

How PDR Works (The Fundamental Equation)

Instead of integrating raw acceleration twice (which leads to extreme noise and error), PDR treats walking as a discrete series of steps. For each detected step k , position is updated in 2D space (x, y) using two parameters: Step Length (L_k) and Heading Angle (θ_k):

[23/09, 11:54 pm] S.....!: $x_k = x_{k-1} + L_k \cdot \sin(\theta_k)$

[23/09, 11:54 pm] S.....!: y_k

[23/09, 11:55 pm] S.....!: Where:

(x_k, y_k) is the new updated position.

(x_{k-1}, y_{k-1}) is the previous position.

L_k is the stride distance of the current step.

θ_k is the heading direction (orientation angle relative to Magnetic North or a reference frame).

The Three Core Modules of PDR

Step Detection: Identifies the precise instant a foot hits the ground (e.g., analyzing cyclic acceleration peaks along the vertical axis).

Step Length Estimation: Determines how far the person traveled during that step. This can be a calibrated fixed distance or dynamically estimated using acceleration magnitude.

Heading Estimation: Measures the direction the person is facing/moving during the step using magnetometers, gyroscopes, or IMUs.

Main Limitation: Error Accumulation (Drift)

Because PDR continually adds relative estimates step after step, errors accumulate over time:

A small 2° magnetic disturbance or gyro bias error shifts every subsequent step off-course.

Sensor noise, hard/soft-iron interference, and gait variations cause the calculated path to drift away from the real physical path.

PDR in the Context of This Paper

In the paper you shared, PDR is implemented on an embedded Arm Cortex-M7 board as follows:

Step Detection: Accelerometer-based thresholding with cooldown filtering.

Step Length: Calibrated fixed step length smoothed using a moving average filter.

Heading: Magnetometer-based 8-direction heading with recoil suppression to prevent false turns.

3D Altitude: Barometric pressure sensor readings added to track vertical changes (elevation/floors) alongside 2D PDR.

[23/09, 11:58 pm] S.....!: In this paper, reverse-direction indications refer to false, temporary heading spikes where the sensor incorrectly registers that the user has suddenly turned around or moved backward for a fraction of a second.

How It Happens (The Physical Cause)

Foot Impact: When a person steps down, the heel contact generates a sharp mechanical shock wave through the body.

Mechanical Recoil: This shock wave causes physical vibration and rapid tilting (recoil) of the sensor unit mounted on the body.

Corrupted Sensor Vector: The magnetometer calculates magnetic North based on its 3D orientation. The violent, short-duration tilt/bounce shifts the sensor's coordinate frame, causing the magnetic field vector math to briefly project a completely inverted heading (a "reverse direction").

How the Paper Fixes It (The Temporal Cooldown Mechanism)

To prevent these false reversals from ruining the trajectory path, the authors implement a temporal cooldown mechanism (time-based lock-out):

Step Event Trigger: When the system detects a step impact, it triggers a brief timer.

Lockout Window: During this short time window immediately following the foot impact, directional heading changes caused by recoil are ignored/suppressed.

Stabilized Heading: Once the physical mechanical shock subsides (a few milliseconds later), directional updates resume normally, preserving stable heading tracking without heavy processing overhead.

[23/09, 11:59 pm] S.....!: Breaking the phrase down word-by-word makes its exact technical meaning clear:

Temporal: Time-based (using a brief delay or cooldown timer).

Suppression: Ignoring, filtering out, or blocking data.

Recoil-induced: Caused by the physical shock and vibration of a foot hitting the ground.

Directional changes: Sudden shifts or spikes in the calculated compass heading.

In Plain English

It means temporarily blocking compass heading updates right after a step is taken, so the system ignores false directional turns caused by foot-impact vibrations.

Step-by-Step Sequence

Step Impact: The user's foot strikes the floor.

Mechanical Recoil: The physical shockwave vibrates the sensor unit, causing a brief, violent tilt.

False Spike: The magnetometer misinterprets this tilt as a sudden change in heading direction.

Temporal Lockout: The system detects the step and starts a tiny timer (a few milliseconds).

Suppression: During this timer window, all incoming heading calculations are thrown away.

Normal Operation Resumes: Once the mechanical shock subsides and the timer expires, heading tracking unlocks and continues cleanly.

[24/09, 12:00 am] S.....!: Human gait analysis is the scientific study and measurement of human walking patterns (how a person moves their legs, feet, and body while walking or

running).

1. What is a "Gait Cycle"?

Walking is a rhythmic, repeating cycle made up of two primary phases for each foot:

Stance Phase (~60% of the cycle): The period when the foot is in contact with the ground.

Heel-strike: The moment the heel touches the floor (causes a high acceleration impact peak).

Foot-flat & Mid-stance: Weight transfers over the stationary foot.

Toe-off: The foot pushes off the ground to propel forward.

Swing Phase (~40% of the cycle): The period when the foot is lifted off the floor and swinging forward through the air.

2. How Inertial Sensors Measure Gait

Wearable sensors (like the accelerometers and gyroscopes on IMUs) capture the distinct physical signals created during this cycle:

Vertical Acceleration Spikes: Used to detect the exact moment of heel-strike (to count steps).

Frequency & Amplitude: Higher, faster peaks correspond to running; lower, slower peaks correspond to casual walking.

Angular Velocity (Gyroscope): Measures leg swing speed and ankle rotation.

3. Why Gait Analysis is Used (Applications)

Pedestrian Navigation (PDR): Helps estimate step length dynamically based on how fast or hard a person is taking steps.

Activity Classification: Differentiates between standing stationary, walking, running, or climbing stairs.

Healthcare & Rehabilitation: Monitors recovery after injuries, detects fall risks in elderly patients, or tracks motor disorders (e.g., Parkinson's disease).

Biometrics & Security: Identifies individuals by their unique walking posture or gait signature.

4. Relevance to This Paper

In the paper, human gait analysis serves as the foundation for the entire system:

It uses the periodic acceleration pattern of the user's gait to trigger step counting.

It analyzes gait dynamics to classify activities (stationary vs. walking vs. running) via the decision tree classifier.

[24/09, 12:01 am] S.....!: Edge Computing and Tiny Machine Learning (TinyML) are two closely related concepts that represent a shift away from traditional, cloud-dependent computing toward processing data directly where it is collected.

1. Edge Computing

Edge Computing is an architectural paradigm where data processing, storage, and AI inference are performed locally on "edge devices" (devices at the edge of the network) rather than sending raw data to remote, centralized cloud servers.

Traditional Cloud AI: Sensor $\rightarrow$ Internet/Wi-Fi $\rightarrow$ Cloud Server (Processes ML) $\rightarrow$ Internet $\rightarrow$ Result sent back to device.

Edge Computing: Sensor $\rightarrow$ Local Device (Processes ML locally) $\rightarrow$ Instant Result.

Edge devices range in scale from industrial gateways and smart cameras to smartphones and embedded boards.

2. Tiny Machine Learning (TinyML)

TinyML is a specialized, ultra-low-power subset of Edge Computing. It focuses specifically on running machine learning models on extremely resource-constrained hardware, such as microcontrollers (MCUs) with:

Processing speeds in MHz (rather than GHz)

Memory measured in kilobytes or a few megabytes (RAM/Flash)

Power budgets measured in milliwatts or microwatts (enabling devices to run on coin-cell batteries for months or years)

Comparison Spectrum

[24/09, 12:02 am] S.....!: FeatureCloud ComputingStandard Edge AITinyML

HardwareServer GPUs/TPUsSmartphones, Gateways, Jetson NanoMicrocontrollers (Arm Cortex-M, ESP32, STM32)

MemoryHundreds of GBsGigabytes (GB)Kilobytes (KB) to a few MB

Power DrawHundreds of WattsSeveral WattsMilliwatts (mW) or Microwatts (μW)

Primary GoalMassive scale, heavy model trainingComplex local processing (e.g., video analytics)Continuous, battery-powered local intelligence

[24/09, 12:02 am] S.....!: Key Advantages of TinyML at the Edge

Zero Network Latency: Decisions happen in milliseconds without waiting for cloud response round-trips.

Minimal Power Draw: Transmitting data wirelessly over Wi-Fi/Bluetooth burns far more battery than running a lightweight model locally on a microcontroller.

Enhanced Data Privacy: Raw motion, biometrics, or audio stay entirely on-device and are never exposed over the network.

Offline Reliability: The system functions continuously in environments with zero internet or GPS coverage (e.g., basements, tunnels, remote areas).

How They Apply to This Paper

In the paper, the authors leverage TinyML and Edge AI to solve a wearable navigation problem:

Hardware: An Arm Cortex-M7 microcontroller (a classic TinyML target platform).

Offline Execution: They trained a Decision Tree classifier on a computer, converted it into lightweight code, and ran it directly on the microcontroller using MicroPython.

Result: The wearable board classifies human physical motion (stationary, walking, running) locally in real time without transmitting raw IMU data to any external server.

[24/09, 12:05 am] S.....!: Here is the detailed explanation of each circled and underlined item from the text, within the context of the paper's navigation algorithm:

1. 100 Hz (Accelerometer Sampling Rate)

What it means: The accelerometer measures body acceleration 100 times per second.

Why it is used in the paper: Human walking and running strides generate rapid impact force peaks during heel-strike that occur over very short timeframes. Sampling at 100 Hz provides enough resolution to cleanly detect these sharp acceleration spikes without missing step events or introducing timing errors.

2. 30 Hz (Magnetometer Continuous-Conversion Rate)

What it means: The magnetic compass sensor updates direction vector readings 30 times per second.

Why it is used in the paper: Human body turning speeds are relatively slow compared to step impacts. Updating magnetic heading at 30 Hz is more than sufficient to track physical turns accurately while saving processing power and battery energy on the microcontroller.

3. Baseline

What it means: The reference static acceleration value measured when the device is completely still at startup.

Why it is used in the paper: When mounted on a user, the sensor is rarely perfectly level; it sits at an angle, measuring a fraction of Earth's gravity ($1g$) across its axes. The baseline value establishes this static tilt reference so it can be subtracted from live readings during movement, isolating pure motion forces.

4. $\text{Bias}_q = \frac{1}{N} \sum_{i=1}^N q_i$, $\quad N=100$ (Baseline Offset Equation)

What it means: The arithmetic average of 100 consecutive stationary readings taken for a given sensor axis q (where q represents the X, Y, or Z axis).

Why it is used in the paper: At a 100 Hz sampling rate, capturing 100 samples takes exactly 1 second during system startup. Averaging 100 static samples cancels out random sensor noise, providing a precise, stable zero-offset value (Bias_q) for each axis before navigation begins.

5. Differential acceleration magnitude between consecutive [samples]

What it means: The change in total acceleration magnitude from one sample to the next:
[24/09, 12:09 am] S.....!: Here is a sentence-by-sentence explanation of each underlined and circled phrase in Image 3, broken down by section:

Section C: Differential Acceleration, Step Detection, and Distance Estimation

1. $TH_{\text{step}} = 180$ and $T_{\text{cooldown}} = 400\text{ ms}$

Sentence Context: "A candidate step is identified when $\Delta M > TH_{\text{step}}$, where $TH_{\text{step}} = 180$. A temporal cooldown interval of $T_{\text{cooldown}} = 400\text{ ms}$ is applied before another step can be registered..."

What & Why:

$TH_{\text{step}} = 180$: Sets a minimum differential acceleration threshold (180 raw ADC counts). The impact of a heel strike must exceed this value to trigger a step, filtering out minor body jitter.

$T_{\text{cooldown}} = 400 \text{ ms}$: Enforces a mandatory 400-millisecond waiting period after detecting a step. Because a human cannot realistically take more than 2.5 steps per second ($1000 \text{ ms} / 400 \text{ ms} = 2.5 \text{ steps/sec}$), this lockout prevents double-counting false peaks during a single stride.

2. $L_{\text{MA}}(k) = \frac{1}{N} \sum_{i=0}^{N-1} L(k-i)$ (Equation 5)

Sentence Context: "For a window containing N detected steps, the smoothed step distance is..."

What & Why: This equation calculates a moving-average step distance over a sliding window of N previous steps. Averaging N consecutive step lengths smooths out step-to-step variations caused by sudden gait changes or sensor noise.

3. Individual gait characteristics

Sentence Context: "...adaptive step-length estimation based on walking speed, activity state, or individual gait characteristics is reserved for future work."

What & Why: Refers to unique personal physical traits like stride length, leg height, and walking posture. The paper currently uses a fixed baseline step length, noting that customizing distances to individual walking styles will be addressed in future research.

Section D: Heading Estimation and Recoil Suppression

4. $\frac{180}{\pi}$ and $(\text{mod } 360)$

Sentence Context: $\theta = \left[\text{atan2}(M_y - V_{\text{offset},y}, M_x - V_{\text{offset},x}) \cdot \frac{180}{\pi} \right] \bmod 360$

What & Why:

$\frac{180}{\pi}$: Standard conversion factor to convert the heading angle returned by atan2 from radians into degrees.

$(\text{mod } 360)$: Ensures the final calculated compass angle stays bounded between 0° and 359.9° .

5. Eight directional

Sentence Context: "The resulting angle is mapped to eight directional sectors: NORTH, NORTH-EAST, EAST, SOUTH-EAST, SOUTH, SOUTH-WEST, WEST, and NORTH-WEST."

What & Why: Quantizes continuous degree angles (e.g., 43.2°) into 8 fixed 45° sectors. Grouping headings into 8 discrete directions simplifies trajectory plotting and

filters out minor heading jitter.

6. Tilt-compensated

Sentence Context: "The present heading implementation is based on calibrated measurements and is not a tilt-compensated three-dimensional compass solution."

What & Why: "Tilt compensation" uses accelerometer angles (pitch/roll) to level out 3D magnetometer readings when the device tilts. The paper clarifies that it does not perform 3D tilt compensation, meaning the sensor board must remain relatively level to maintain accurate compass directions.

Section E: Barometer Estimation

7. Barometer captures minute [localized changes in atmospheric pressure]

Sentence Context: "The barometer captures minute localized changes in atmospheric pressure to correct these vertical errors down to centimeters."

What & Why: Atmospheric pressure drops systematically with height. High-precision barometric pressure sensors can detect tiny air pressure shifts, allowing the system to track indoor floor changes (altitude/staircases) with centimeter-level accuracy where GPS fails.

Section F: Offline TinyML Activity Classification

8. Sensor-derived motion features and heading information

Sentence Context: "A labeled dataset of 1500 samples is collected using sensor-derived motion features and heading information."

What & Why: Refers to the input dataset used to train the machine learning model. Instead of feeding raw data directly, calculated metrics like movement variance and heading changes were compiled into 1,500 labeled records.

9. Tree classifier

Sentence Context: "A Scikit-Learn Decision Tree classifier with a maximum depth of three is trained offline on a PC."

What & Why: Specifies the machine learning algorithm chosen. A Decision Tree classifies data using basic branching rules (if feature > value). Restricting depth to 3 keeps the model compact enough to run on embedded hardware.

10. Conditional statements & Approach reduces computational and memory overhead

Sentence Context: "The trained decision structure is translated into explicit conditional statements... This approach reduces computational and memory overhead..."

What & Why:

Conditional statements: The trained PC decision tree is converted directly into native if-elif-else code blocks in MicroPython.

Reduces overhead: Running plain if statements avoids needing heavy machine learning math libraries on the microcontroller, saving CPU processing cycles and system memory.

Section III-C: Computational Performance

11. Latency is less than 0.1 ms per frame & 15 KB of RAM

Sentence Context: "...the reported classification latency is less than 0.1 ms per frame on the Cortex-M7 core. The reported memory overhead is below 15 KB of RAM."

What & Why:

Latency < 0.1 ms: Processing a data frame and deciding whether the user is walking, running, or standing takes less than 0.0001\text{ seconds}, providing instant real-time response.

15 KB of RAM: The entire ML algorithm uses less than 15 kilobytes of RAM, demonstrating extreme efficiency suitable for ultra-low-power microcontrollers.

[24/09, 12:11 am] S.....!: Here is the detailed, sentence-by-sentence explanation of each circled and underlined term in Image 4, in the context of the paper's indoor navigation system:

1. PDR (Pedestrian Dead Reckoning)

Context in Paper: "Table III summarizes comparative benchmarks between standard fixed-threshold PDR and the proposed adaptive framework."

What & Why: Refers to the standard, non-AI baseline method of tracking step movement. The paper uses traditional PDR as a benchmark to demonstrate how adding its TinyML decision tree significantly reduces distance drift (dropping error from 12.4%–18.2% down to 2.8%–4.1%) and reduces false step triggers.

2. confusion matrix

Context in Paper: "The activity-classification confusion matrix also indicates high separation among stationary, walking, and running states."

What & Why: A machine learning performance matrix (shown in Table II) that explicitly breaks down correct vs. incorrect classifications. It proves that the model almost never confuses standing still with walking or running, achieving a high overall accuracy of 98.2\%.

3. moving-average filter removes air-draft spikes from vertical floor tracking

Context in Paper: "...while the moving-average filter removes air-draft spikes from vertical floor tracking."

What & Why: High-precision barometers are sensitive enough to detect minute air pressure changes caused by sudden HVAC airflow, opening doors, or wind drafts. These transient pressure jumps can be misinterpreted by the system as moving up or down a floor. Applying a 3-point moving-average filter smooths out these rapid pressure spikes, ensuring stable vertical altitude/floor tracking.

4. lack of three-dimensional tilt compensation

Context in Paper: "The principal limitation of the current directional implementation is its susceptibility to environmental magnetic interference and its lack of three-dimensional tilt compensation."

What & Why: If a compass sensor tilts (pitch or roll), the measured 2D magnetic vector gets distorted, giving an inaccurate heading. The authors acknowledge that their current system requires the sensor module to remain level, and that dynamic tilt correction using accelerometer angle calculations is left for future work.

5. Hard-iron and soft-iron effects

Context in Paper: "Hard-iron and soft-iron effects, as well as nearby metallic structures, can alter the measured magnetic field and consequently affect heading accuracy."

What & Why: Magnetic distortions that corrupt compass headings indoors:

Hard-iron effects: Permanent magnetic fields created by components on the PCB itself or nearby magnetized metals, which add a constant shift/bias to sensor readings.

Soft-iron effects: Temporary field distortions caused by external ferromagnetic materials in buildings (like steel structural beams, elevator shafts, or iron doors) that bend Earth's local magnetic field.

6. work rather than a complete indoor positioning solution (and handwritten "indoor mapping?")

Context in Paper: "The current work should therefore be regarded as a lightweight motion-sensing and activity-classification framework work rather than a complete indoor positioning solution."

What & Why: Sets realistic scope boundaries for the research. Because relative dead reckoning accumulates small errors over long distances, this system serves as a low-power sensing foundation rather than a standalone, long-term indoor GPS replacement (which would require external anchoring like Bluetooth beacons or Wi-Fi signals to reset cumulative drift).

7. 2D/3D mapping

Context in Paper: "This paper presented a fully offline Edge-AI framework for step counting, heading estimation, distance estimation, and activity classification, 2D/3D mapping using a crossover processor..."

What & Why: Refers to reconstructing a user's full spatial trajectory:

2D: Horizontal movement (X, Y floor coordinates calculated via accelerometer and magnetometer).

3D: Vertical altitude and multi-floor tracking (Z coordinate calculated via barometric pressure).

8. 15 KB of RAM

Context in Paper: "The deployed Decision Tree also provided a reported classification latency below 0.1 ms with memory overhead below 15 KB of RAM."

What & Why: Re-emphasizes the TinyML efficiency of the implementation. Utilizing less than 15\text{ KB} of system memory confirms that the entire navigation and classification software can comfortably execute on small, low-cost microcontrollers without starving the processor of resources.